Multi-Agent Orchestration for RAG Systems

Step 1. RAG Baseline Reproduction & Learning Report

Goal: Reproduce baseline results with BM25, Dense Retrieval, GraphRAG, Hybrid Retrieval, and re-ranking, while reporting key metrics (Precision@k, Recall@k, MRR). Date: May 2026 | Environment: Google Colab / Local Python | Notebook: ***Step_1_Baseline_and_Failure_Analysis.ipynb***

Part 1: Project Flow & Concept Learnings

Before looking at the results, we wanted to really understand how the original system was built. Overall, the original RAG (Retrieval-Augmented Generation) setup essentially works like an "open-book" search engine. It reads through the project's ETH Zurich document corpus to find the best paragraphs to answer specific questions.

The system uses three different search agents for this:

- **BM25 (The Keyword Agent):** This focuses on exact text matches. It simply counts how often rare words appear to rank the documents.
- **Dense Retriever (The Meaning Agent):** This uses Hugging Face models (`multilingual-e5-large-instruct`) to turn text into vectors. This helps the system understand synonyms and the overall meaning instead of just exact words.
- **GraphRAG (The Detective Agent):** This uses a Knowledge Graph to find hidden connections across different documents.

The Orchestrator acts as the manager. It uses strategies like Voting, Waterfall, or Confidence to decide which agents to rely on. Then, it combines their results mathematically using Reciprocal Rank Fusion (RRF).

1.1 Reproducibility Architecture Instead of dealing with a bunch of scattered Python files from the original project, our colleague built custom "Adapter" classes. This gives all three agents the exact same `search(query, top_k)` interface. Because of this, it doesn't matter how the original data was saved (like via LangChain or plain Python). Everything loads properly now.

A random seed was also locked globally to ensure that metrics do not fluctuate across different runs:

```
import random, numpy as np, os

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)
```

Part 2: Results

(The sections below keep the original comments and tables directly from the reproduction notebook.)

> *This notebook reproduces baseline retrieval results for https://github.com/Trista1208/advanced_genAI.git (23.12.2025). The reported configuration uses the full corpus. We load BM25, Dense, and GraphRAG resources for the full corpus, then construct Hybrid and Re-ranking on top of the same retrieved candidates using consistent fusion and reranking logic. We recompute all metrics ourselves from the benchmark QA set and qrels with one shared evaluator, reporting Precision@k, Recall@k, and MRR identically for every method. This setup ensures reproducibility in our environment and keeps comparisons fair across methods on the realistic full-corpus setting.*

1.3 Evaluation Metrics Here is a quick refresher on what we are measuring: - **Precision@K:** Out of the top K documents the system gave us, how many were actually correct? - **Recall@K:** Out of all the correct documents out there, how many did the system actually find in its top K results? - **MRR (Mean Reciprocal Rank):** This looks at the position of the first correct answer. If the first relevant document is at rank 1, the score is 1.0. If it's further down, the score drops.

2.1 Baseline Evaluation (Full Corpus)

This cell runs the shared evaluation pipeline for all baseline methods using the same QA set and qrels. For each query, it collects ranked document IDs, computes Precision@k, Recall@k, and reciprocal rank, and then aggregates results into per-query and per-method summary tables. Using one evaluation function for all methods ensures the reported baseline comparison is consistent and directly comparable.

method	queries_evaluated	MRR	Precision@1	Recall@1	Precision@3	Recall@3	Precision@5	Recall@5	Precision@10	Recall@10
GraphRAG	24	**0.232573**	0.083333	0.000687	0.097222	0.003166	0.116667	0.005892	0.116667	0.052857
ReRank	24	0.222952	0.041667	0.000196	0.138889	0.004480	0.125000	0.006323	0.100000	0.010619
Hybrid	24	0.202154	0.000000	0.000000	0.097222	0.003699	0.125000	0.028352	0.095833	0.031149
Dense	24	0.165525	0.041667	0.000147	0.069444	0.008953	0.058333	0.010095	0.066667	0.034097
BM25	24	0.151296	0.041667	0.001016	0.055556	0.001681	0.091667	0.005506	0.091667	0.011165

On the full-corpus setup, GraphRAG achieved the best overall ranking quality with the highest MRR (0.233), indicating that graph-guided retrieval was most effective at placing relevant evidence early in the ranked list. ReRank and Hybrid followed closely, with ReRank slightly outperforming Hybrid on MRR, which suggests that post-fusion refinement can improve early precision in some cases. Dense and BM25 showed lower MRR, indicating weaker top-rank relevance when used alone in this setting. Overall, the results suggest that full-corpus retrieval benefits from multi-source or graph-aware methods, while single-retriever baselines remain useful reference points but are less competitive at early-rank retrieval quality.

2.2 Orchestration Evaluation (Full Corpus) The three orchestration strategies are evaluated separately from the individual baselines to keep the comparison clean.

- **Confidence:** Analyzes the query first, then weights agents by question type (keyword-heavy vs. semantic).
- **Waterfall:** Starts with only BM25 and Dense. Adds GraphRAG only when the two disagree significantly.
- **Voting:** Runs all three agents in parallel and merges results using weighted Reciprocal Rank Fusion (RRF).

method	queries_evaluated	MRR	Precision@1	Recall@1	Precision@3	Recall@3	Precision@5	Recall@5	Precision@10	Recall@10
Confidence	24	**0.208827**	0.000000	0.000000	0.111111	0.003891	0.133333	0.028380	0.100000	0.031569
Waterfall	24	0.208303	0.041667	0.001344	0.138889	0.005348	0.100000	0.006052	0.070833	0.029732
Voting	24	0.202154	0.000000	0.000000	0.097222	0.003699	0.125000	0.028352	0.095833	0.031149

For orchestration on the full corpus, Confidence achieved the best MRR (0.209), with Waterfall very close (0.208) and Voting slightly lower (0.202), so overall early-rank performance is similar across all three strategies. Waterfall produced the strongest Precision@3 (0.139), indicating better short-list relevance in the top few results, while Confidence led at Precision@5 (0.133). At larger

cutoffs, Confidence and Voting were nearly tied on Recall@10, with Waterfall slightly behind. In practice, these results suggest that orchestration variants are competitive but not dramatically separated, with Confidence showing the most balanced behavior and Waterfall favoring early precision at small k.

2.3 Reported Scope The reported Step 1 reproduction uses the **full corpus** (7,531 chunks). Older subsample-compatible code remains in the notebook only to support legacy artifact formats, but subsample results are not part of the current reported baseline. We use the full corpus because it is the realistic retrieval setting for the downstream reliability experiments.

2.4 Reproducibility Comparison Table

Aspect	Original Report (PDF)	Reproduced Notebook (Current)	Match Status
Full-corpus size	7,544 docs/chunks	7,531 fixed-size chunks (local artifacts)	Minor mismatch
Full-corpus Dense MRR	0.166 (reported best baseline)	0.166	Match (value)
Full-corpus baseline ranking	Dense reported strongest baseline	GraphRAG/ReRank/Hybrid above Dense in reproduced run	Mismatch
Full-corpus orchestration best	Confidence ~ 0.205 (Step 3)	Confidence ~ 0.209	Close match
Full-corpus Voting MRR	~ 0.190 (Step 3), ~ 0.189 (Step 2 section)	~ 0.202	Near but higher
Full-corpus Waterfall MRR	~ 0.161 (Step 3)	~ 0.208	Mismatch
Evaluation protocol	Shared IR metrics (P@k, Recall, MRR, plus nDCG in report)	Shared IR metrics (P@1/3/5/10, Recall@1/3/5/10, MRR)	Largely aligned

Our reproduction matches key full-corpus numbers such as Dense MRR (0.166). Some of the differences in specific scores or rankings are probably because we use different artifact versions or slightly different settings for fusion and reranking.

2.5 Answer Quality and Efficiency Findings After choosing **Confidence** as the Step 1 reference baseline, we also evaluated answer synthesis with the previous-semester `step2_llm` Mistral-style generation path. This is important because retrieval metrics alone do not show whether the final generated answer is actually correct.

For the reference baseline (`Confidence`, 24 benchmark queries), retrieval quality was modest: `MRR=0.209`, `Precision@5=0.133`, and `Recall@10=0.032`. This means that relevant evidence was often not ranked high enough for the answer synthesis step. Answer quality was also limited: `Answerable@5` was `0.50`, the median first relevant rank was `5`, average token-F1 against the gold answers was `0.184`, and exact match was `0.0`.

The qualitative examples showed three important patterns. First, several generated answers were fluent but focused on related evidence rather than the exact target answer, such as the ERC grants and famous ETH alumni examples. Second, when relevant evidence was missing from the top retrieved context, the system either abstained with `NOT FOUND IN CONTEXT` or produced plausible but unsupported answers. Third, even when the answer was semantically correct, automatic metrics could still be harsh: for the question "when did the insight get to mars?", the generated answer included "November 26, 2018", while the gold answer was "26 november 2018", so the answer received `exact_match=0` despite matching the date.

System efficiency was also a practical limitation. The Mistral synthesis path had a mean latency of about `76.16s` per query and a p95 latency of about `85.06s` per query in our run. The approximate cost proxy is therefore `Medium/High`, because every query requires LLM decoding on top of retrieval.

Overall, today's answer-quality findings confirm that the Step 1 baseline is useful but not reliable enough as a final system. The main weaknesses are weak evidence ranking, answers that can focus on related but wrong evidence, unsupported generation when context is insufficient, and high generation latency. Metric brittleness is a secondary evaluation issue, not the main system weakness. These findings directly motivate the later reliability mechanisms in Steps 2-4, especially evidence sufficiency checks, groundedness checks, abstention, critique, and recovery.

Conclusion

By putting all the original project's scripts into one clean Jupyter Notebook, we now have a really solid and reproducible baseline. Out of all the single agents, GraphRAG did the best on the full corpus (MRR 0.233). For the orchestrators, Confidence came out on top (MRR 0.209).

The biggest takeaway is that full-corpus retrieval remains difficult and answer synthesis remains fragile: even the best retrieval methods have modest early-rank performance, and the generated answers do not consistently match the gold answers. This is why the later project stages focus not only on retrieving evidence, but also on deciding whether the evidence is sufficient and whether the system should answer, recover, clarify, or abstain.